

Trabajo previo · Taller 02

Caso aplicado de IA en industria II Programa de Especialización en IA Generativa y Machine Learning Ops · UNI

Sábado 8 de agosto, 15:00–19:00 · Domingo 9 de agosto, 09:00–13:00

Léeme primero

Este taller no empieza con diapositivas. Empieza con **192 modelos corriendo en tu máquina**, y uno de ellos portándose mal.

Para que eso sea posible el sábado a las 15:00, necesitas dejar el entorno listo **antes**. Entre 20 y 30 minutos, casi todos de descarga.

Por qué importa hacerlo antes y no ese mismo día: la primera vez, Docker construye las imágenes y baja varios cientos de megas de dependencias. Si veinte personas lo hacemos a la vez sobre el wifi del aula, la primera hora se nos va mirando barras de progreso. Y esa hora es la del caso.

Lo que vas a construir

Una cadena de retail con 24 tiendas y 8 categorías pronostica su demanda todas las noches. Un modelo por tienda y categoría: **192 modelos en producción**. Cada madrugada un job los carga, proyecta los próximos 14 días, y con eso se decide qué reponer.

Los modelos funcionan. Hasta que dejan de funcionar.

Vas a construir un **agente generativo con arquitectura cognitiva** que vigila esa flota: percibe, recuerda, razona, se cuestiona a sí mismo, y actúa.

Los 192 modelos vienen dados: son *tu producción*. **No vas a tocarlos**. Lo que vas a construir es el agente que los vigila.

0 · Lo que necesitas

Requisito	Detalle
Docker Desktop	Con Docker Compose (viene incluido)
Disco	12 GB libres. Es bastante; más abajo explico por qué y cómo recuperar 4 GB al terminar
Tiempo	20–30 minutos, casi todos de descarga
Docker abierto	No basta con instalarlo: tiene que estar corriendo

No necesitas Python instalado. No necesitas GPU. No necesitas WSL2. No necesitas saber Docker más allá de copiar y pegar.

Dos formas de dar los comandos

El laboratorio se maneja con comandos cortos, y hay una versión para cada sistema. **Hacen exactamente lo mismo** — corren los mismos contenedores y producen los mismos resultados.

Tu sistema	Escribes	Terminal
Windows	<code>.\taller.ps1 arriba</code>	PowerShell
macOS o Linux	<code>make arriba</code>	Terminal

En este documento verás las dos. Usa la de tu sistema e ignora la otra.

Si usas Windows, sigue a la sección 1. Si usas macOS o Linux, salta a la sección 2.

1 · Windows: dos minutos de preparación

Si usas macOS o Linux, esta sección no va contigo. Salta a la 2.

1.1 · Instala Docker Desktop

Descarga [Docker Desktop](#) e instálalo con las opciones por defecto. Ábrelo y **espera a que el ícono de la ballena deje de moverse** — mientras se mueve, todavía está arrancando.

Si la instalación falla diciendo algo sobre virtualización, hay que habilitarla en el BIOS de tu equipo. Búscala como *Virtualization*, *VT-x* (Intel) o *SVM* (AMD). Si no te animas a tocar el BIOS, escíbeme al foro.

1.2 · Permite que PowerShell corra el script del taller

Windows bloquea por defecto los scripts de PowerShell. Abre **PowerShell** y corre esto una sola vez:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Responde **S** (o **Y**) cuando pregunte.

Esto solo afecta a tu usuario y permite correr scripts locales como el del taller. Si prefieres no cambiar la política, puedes invocar cada comando así:

```
powershell -ExecutionPolicy Bypass -File .\taller.ps1 arriba
```

Es lo mismo, pero más largo de escribir cada vez.

1.3 · Eso es todo

No necesitas WSL2, ni instalar **make**, ni una terminal de Linux. Todo el taller corre desde PowerShell.

2 · Verifica Docker

Abre tu terminal — **PowerShell** en Windows, la Terminal en macOS o Linux — y corre:

```
docker --version
docker compose version
git --version
```

Las tres deben responder con un número de versión.

Si **docker compose version** responde **unknown command**, prueba con guion, que es la versión antigua:

```
docker-compose --version
```

Si responde, estás bien: el laboratorio detecta solo cuál de las dos tienes.

Si **docker** no responde nada, Docker Desktop no está instalado o no está abierto. Ábrelo y espera a que termine de arrancar.

3 · Levanta el entorno

Clona el repositorio:

```
git clone https://github.com/texai/taller-ia-uni-lab.git
cd taller-ia-uni-lab
```

En Windows (PowerShell):

```
Copy-Item .env.example .env
.\taller.ps1 arriba
.\taller.ps1 seed
```

En macOS o Linux:

```
cp .env.example .env
make arriba
make seed
```

Qué hace cada cosa:

- **arriba** construye las tres imágenes la primera vez y levanta la plataforma. **Entre 4 y 6 minutos** con buena conexión — es el paso largo, y la razón principal para hacer esto antes del sábado.
- **seed** genera el histórico de ventas, entrena los 192 modelos, corre el job de pronóstico y calcula las métricas. Menos de un minuto.

Al final debes ver exactamente esto:

```
Listo. 192 modelos con 17,472 dias-modelo de telemetria.
```

Si ves ese mensaje, la parte pesada ya pasó.

Sobre el espacio en disco

Las tres imágenes pesan alrededor de **6 GB**, y la construcción deja otros **~4 GB de caché**. Es bastante, y la razón es que el laboratorio trae la plataforma de ML completa —scikit-learn, MLflow, pandas— más la pila de agentes con los cinco proveedores de modelo, para que uses el que prefieras sin reconstruir nada.

Una vez que **seed** terminó bien, puedes recuperar la caché:

```
docker builder prune -f      # libera ~4 GB, no toca lo que ya construiste
docker system df             # para ver en qué estás
```

4 · Consigue una llave de modelo

El agente necesita un modelo de lenguaje. Es agnóstico al proveedor: eliges uno y pones su llave.

Abre el archivo **.env** que copiaste y completa **solo las líneas de tu proveedor**. Es un archivo de texto: ábrelo con el Bloc de notas, VS Code, o lo que uses. En Windows, **notepad .env** desde PowerShell te sirve.

Opciones gratuitas — recomendadas

Google AI Studio → aistudio.google.com Llave instantánea con tu cuenta de Google, sin tarjeta.

```
PROVEEDOR_LLM=google
MODELO_LLM=gemini-2.0-flash
GOOGLE_API_KEY=tu-llave-aca
```

Groq → console.groq.com Llave instantánea, muy rápido.

```
PROVEEDOR_LLM=groq
MODELO_LLM=llama-3.3-70b-versatile
GROQ_API_KEY=tu-llave-aca
```

Si ya pagas OpenAI o Anthropic

También funcionan. Cada corrida del agente son unas diez llamadas a herramientas y tarda menos de un minuto, así que el gasto de las dos sesiones es modesto — pero si te preocupa el costo, usa una de las gratuitas de arriba, que para este taller rinden igual.

```
PROVEEDOR_LLM=anthropic
ANTHROPIC_API_KEY=tu-llave-aca
```

Si prefieres no usar ninguna nube

```
.\taller.ps1 ollama      # Windows
make ollama             # macOS o Linux
```

Descarga ~2 GB, así que hazlo antes del sábado. Y en `.env` : `PROVEEDOR_LLM=ollama`

Funciona, pero con un modelo local pequeño el agente razona notoriamente peor. Sirve para seguir la clase; no para ver de lo que es capaz.

Último recurso

`PROVEEDOR_LLM=mock` recorre toda la arquitectura sin llamar a ningún modelo. No razona, pero **no te deja trabado**: si tu llave falla el sábado, con esto sigues la clase mientras lo resolvemos.

5 · Verifica que funciona

Abre **`http://localhost:8501`** en tu navegador.

Debes ver:

- Un mensaje verde: *Entorno listo · 192 modelos en produccion · 17,472 dias-modelo de telemetria*
- Cuatro indicadores arriba: Modelos, MAPE medio, Sesgo, Cobertura
- Un gráfico de MAPE por categoría, con ocho líneas

- Una tabla con las peores filas de telemetría

Y para comprobar que **todo** el laboratorio está sano, incluida tu llave:

```
.\taller.ps1 verificar      # Windows  
make verificar             # macOS o Linux
```

Debe terminar con:

```
Las 24 comprobaciones pasaron.
```

Tarda alrededor de un minuto y no consume tokens de tu llave, salvo una única llamada corta para confirmar que el proveedor responde.

Si ves las 24 en verde, estás listo para el sábado.

6 · Si algo falla

Lo que ves	Qué significa	Qué hacer
<code>no se puede cargar el archivo taller.ps1</code>	Windows bloquea los scripts	Vuelve al paso 1.2
<code>unknown command: compose</code>	Falta Docker Compose	Instala Docker Desktop
<code>Docker Desktop no esta corriendo</code>	Justo eso	Ábrelo y espera a que la ballena deje de moverse
<code>open //./pipe/dockerDesktopLinuxEngine</code>	Lo mismo, dicho de forma críptica	Ábrelo y espera a que la ballena deje de moverse
<code>Cannot connect to the Docker daemon</code>	Lo mismo, en macOS o Linux	Abre Docker Desktop y espera
La instalación de Docker falla	Virtualización deshabilitada	Actívala en el BIOS: <i>Virtualization</i> , <i>VT-x</i> o <i>SVM</i>
<code>no space left on device</code>	Disco lleno	<code>docker builder prune -f</code> libera ~4 GB sin tocar tus imágenes. Si no alcanza: <code>docker system prune -a</code>
<code>port is already allocated</code>	Otro proceso usa el 8000 o el 8501	Ciérralo, o avísame en el foro
La UI dice <i>No se pudo contactar la plataforma</i>	La plataforma no terminó de levantar	Espera 30 s y recarga. Si sigue, mira el estado
La UI dice <i>no hay telemetria todavia</i>	Falta el paso de datos	Corre <code>seed</code>
El agente responde con <code>[modelo simulado]</code>	No hay llave configurada	Revisa <code>PROVEEDOR_LLM</code> en tu <code>.env</code>

Dos comandos que sirven para casi todo, más el botón de pánico:

Qué hace	Windows	macOS / Linux
Qué contenedores corren	<code>.\taller.ps1 estado</code>	<code>make estado</code>
Qué dijeron	<code>.\taller.ps1 logs</code>	<code>make logs</code>
Borra todo y reconstruye	<code>.\taller.ps1 reset</code>	<code>make reset</code>

Si nada de esto lo arregla, **escribe al foro antes del viernes** con:

1. Tu sistema operativo
2. La salida de `docker compose version`
3. El error exacto, copiado y pegado

Lo que NO necesitas hacer

- **No necesitas leer el código** antes de la clase. Lo recorreremos juntos.
 - **No necesitas saber Docker.** Los comandos del taller son diez.
 - **No necesitas WSL2**, ni Python, ni GPU.
 - **No necesitas repasar estadística.** Lo que haga falta lo vemos en su momento, aplicado.
 - **No hay entregable ni examen.** El taller se evalúa por asistencia.
-

Referencia rápida

Qué hace	Windows	macOS / Linux
Lista los comandos	<code>.\taller.ps1</code>	<code>make</code>
Levanta el entorno	<code>.\taller.ps1 arriba</code>	<code>make arriba</code>
Lo apaga	<code>.\taller.ps1 abajo</code>	<code>make abajo</code>
Pipeline completo	<code>.\taller.ps1 seed</code>	<code>make seed</code>
Comprueba todo	<code>.\taller.ps1 verificar</code>	<code>make verificar</code>
Abre la interfaz	<code>.\taller.ps1 ui</code>	<code>make ui</code>
Qué está corriendo	<code>.\taller.ps1 estado</code>	<code>make estado</code>
Borra todo y reconstruye	<code>.\taller.ps1 reset</code>	<code>make reset</code>

Repositorio: <https://github.com/texai/taller-ia-uni-lab>

Una nota sobre los datos

El histórico es **sintético** y la cadena es ficticia. Pero la mecánica de degradación reproduce modos de falla reales de sistemas de pronóstico en producción, y los umbrales del taller están medidos contra esa flota, no inventados.

Lo que aprendas acá es transferible: el agente sirve para vigilar cualquier flota de modelos que emita predicciones evaluables contra la realidad.

Nos vemos el sábado.

Ernesto Anaya